GAME PROGRAMMING COURSE — DESIGN PATTERN

シングルトン深堀り

SINGLE
TON 3

作って、わざと壊して、使い分けを体で覚える

全員で一緒に打ち込む回

今日作るもの

ステージのタイムと、ベストタイム



作るクラス

- **StageTimer**：今のステージのタイムを計る
- **SaveDataManager**：セーブするゲームデータを管理（今日は**ベストタイム**）
- チャレンジ：**PauseMenu**（Esc でポーズ）

POINT どちらも「時間」を扱うのに、**片方は Scene、片方は Persistent**になる。その違いが今日の主役。

使い分けの判断

迷ったら、この2つの質問だけ

	Q2　シーン内の物を触る	Q2　触らない
Q1　シーンが変わっても値を残したい	2つに分ける データ＝Persistent、表示＝Scene	Persistent
Q1　残さなくていい	Scene	Scene（シングルトンでなくてもよい）

POINT　Q1 は「**リトライやタイトルに戻っても**覚えていてほしいか」。Q2 は「Inspector に**シーン内の物をドラッグ**するか」で考えると早い。

確認

前回のクラスを2つの質問にあてはめる

クラス	Q1 残したい？	Q2 シーン内を触る？	結果
SceneLoadManager	はい（ロード中も動く）	いいえ（自分の Canvas）	Persistent
AudioManager	はい（BGM を流し続ける）	いいえ	Persistent
GameDataHolder	はい（クリア状況）	いいえ	Persistent
InGameUi	いいえ	はい（シーンの UI）	Scene
TPSCameraController	いいえ	はい（Player を追う）	Scene

STEP 1

StageTimer

ステージのタイムを計って表示する

1

STEP 1　判断してみよう

StageTimer はどちらにする？

ステージが始まってからの時間を計り、画面の Text に表示する。

質問	StageTimer は？
Q1　シーンが変わっても値を残したい？	？
Q2　シーン内の物（UI・Player など）を触る？	？

STEP 1　答え

StageTimer は Scene

質問	StageTimer は？
Q1　シーンが変わっても値を残したい？	いいえ （リトライしたら 0 から）
Q2　シーン内の物（UI・Player など）を触る？	はい （画面の Text）

POINT

リトライで**0 に戻ってほしい**し、**シーンの Text** を触る。だから **SceneSingleton**。シーンと一緒に作り直されるので、リセットの処理を書かなくてよい。

STEP 1　準備

Unity でやること

- ① ステージのシーン（例：Tutorial_0）を開く
- ② ゲーム中の Canvas に **TextMeshPro - Text** を追加し、名前を **TimeText**（左上に配置）
- ③ 空の GameObject「**StageTimer**」を作る（スクリプトは次のページで書く）
- ④ 書けたら StageTimer に付けて、**timeText** に TimeText をドラッグ

POINT　Inspector に**シーン内の物をドラッグ**している ＝ Q2 が「はい」。Scene を選んだ理由そのもの。

StageTimer.cs（STEP 1）

C# Assets/Scripts/StageTimer.cs

```
using TMPro;
using UnityEngine;

// ステージのクリアタイムを計る（シーンごとにリセット）
public class StageTimer : SceneSingleton<StageTimer>
{
    [SerializeField] private TextMeshProUGUI timeText;

    private float elapsedTime;
    private bool isRunning = true;

    private void Update()
    {
        if (!isRunning) return;
        elapsedTime += Time.deltaTime;
        timeText.text = "TIME " + elapsedTime.ToString("F2");
    }

    public void StopTimer()
    {
        isRunning = false;
    }
}
```

STEP 1　解説

Update で時間を足して表示する

C# Assets/Scripts/StageTimer.cs

```
12      private void Update()
13      {
14          if (!isRunning) return;
15          elapsedTime += Time.deltaTime;
16          timeText.text = "TIME " + elapsedTime.ToString("F2");
17      }
```

ポイント

- **Time.deltaTime**：前のフレームからの秒数
- 足していくと**経過時間**になる
- **ToString("F2")**：小数2桁の文字列

STEP 1　動かしてみよう

ここまでで確かめること

1. 再生すると、左上の TIME が**増えていく**
2. わざとやられる → Enter でリトライ → TIME が **0.00 から**始まる
3. Hierarchy を見る：StageTimer は **DontDestroyOnLoad の中にはない**

STEP 2

タイマーを止める

2

やられた・クリアしたら止める（Player.cs に1行ずつ追記）

STEP 2　追記

Player.GameOver() に1行足す

C# Assets/Scripts/Player.cs

```
140     public void GameOver()
141     {
142         if (isGameEnd) return;
143         isGameEnd = true;
144         anim.SetFloat("Speed", 0f);
145         Break();
146         InGameUi.Instance.GameOver();
147         TPSCameraController.Instance.ShakeCam();
148         StageTimer.Instance.StopTimer();  // 追加
149     }
```

ポイント

- 最後に **StageTimer.Instance.StopTimer();**
- Player は**シーン内の物**、StageTimer も**シーン内**
- 呼ぶのは Start 以降なので順番の心配なし

STEP 2　追記

Player.GameClear() にも1行足す

C# Assets/Scripts/Player.cs

```
171     public void GameClear()
172     {
173         if (isGameEnd) return;
174         isGameEnd = true;
175         anim.SetFloat("Speed", 0f);
176         InGameUi.Instance.GameClear();
177         StageTimer.Instance.StopTimer();  // 追加
178     }
```

ポイント

- クリアしたときも**止める**
- この行は STEP 3 で**書き換える**

POINT　再生して、ゴールしたら TIME が**止まる**ことを確認。

STEP 3

SaveDataManager

3

セーブするゲームデータを管理する（まずはベストタイム）

STEP 3　判断してみよう

SaveDataManager はどちらにする？

セーブしたいゲームデータをまとめて管理する。今日はその1つ目として、ステージごとのベストタイムを入れる。

質問	SaveDataManager は？
Q1　シーンが変わっても値を残したい？	？
Q2　シーン内の物（UI・Player など）を触る？	？

POINT　**まだ書かない。**隣の人と30秒話して、Scene か Persistent かを決めてから次へ。

STEP 3　答え

SaveDataManager は Persistent

質問	SaveDataManager は？
Q1　シーンが変わっても値を残したい？	はい （リトライしても タイトルに戻っても）
Q2　シーン内の物（UI・Player など）を触る？	いいえ （数字を持つだけ）

POINT

シーンが変わっても**覚えていてほしい**。シーン内の物は触らない。だから **PersistentSingleton**。表示は StageTimer（Scene）に任せる ＝ 表の左上「**2つに分ける**」。

STEP 3　準備

Dictionary — 名前で値を引ける入れ物

C# Dictionary の使い方

```
Dictionary<string, float> bestTimes = new Dictionary<string, float>();

bestTimes["Tutorial_0"] = 12.3f;     // 入れる・上書き
float t = bestTimes["Tutorial_0"];   // 取り出す
bool has = bestTimes.ContainsKey("Tutorial_1");  // ある？

// あれば true と値、なければ false
if (bestTimes.TryGetValue("Tutorial_0", out float best)) { }
```

キー（シーン名）	値（ベストタイム）
Tutorial_0	12.30
Tutorial_1	18.75
FirstIngame_0	（まだ記録なし）

SaveDataManager.cs（STEP 3）

C# Assets/Scripts/SaveDataManager.cs

```
using System.Collections.Generic;
using UnityEngine;

// セーブするゲームデータをまとめて管理する（シーンをまたいで残る）
// 今日はベストタイムだけ。ほかのデータもここに増やしていく
public class SaveDataManager : PersistentSingleton<SaveDataManager>
{
    // ===== ベストタイム =====
    // キー：シーン名　値：ベストタイム（秒）
    private Dictionary<string, float> bestTimes = new Dictionary<string, float>();

    /// <summary>
    /// ゲーム起動時に生成
    /// </summary>
    [RuntimeInitializeOnLoadMethod]
    private static void Init()
    {
        SaveDataManager pref = Resources.Load<SaveDataManager>("Singletons/SaveDataManager");
        if (pref != null) Instantiate(pref);
    }
```

SaveDataManager.cs（STEP 3）

C# Assets/Scripts/SaveDataManager.cs

```
22      // 記録を更新できたら true を返す
23      public bool TryUpdateBestTime(string sceneName, float time)
24      {
25          if (bestTimes.TryGetValue(sceneName, out float best) && best <= time)
26          {
27              return false;
28          }
29          bestTimes[sceneName] = time;
30          return true;
31      }
32
33      public bool HasBestTime(string sceneName)
34      {
35          return bestTimes.ContainsKey(sceneName);
36      }
37
38      public float GetBestTime(string sceneName)
39      {
40          return bestTimes[sceneName];
41      }
42  }
```

STEP 3　解説

TryUpdateBestTime — 速かったときだけ書き換える

C# Assets/Scripts/SaveDataManager.cs

```
23      public bool TryUpdateBestTime(string sceneName, float time)
24      {
25          if (bestTimes.TryGetValue(sceneName, out float best) && best <= time)
26          {
27              return false;
28          }
29          bestTimes[sceneName] = time;
30          return true;
31      }
```

- 記録があって、**前の方が速い**（**同じ**）なら false
- それ以外（**初記録**か**更新**）は書き込んで true
- true が返ったら「NEW RECORD!」を出せる

STEP 3　解説

Persistent はプレハブから起動時に作る

C# Assets/Scripts/SaveDataManager.cs

```
12      /// <summary>
13      /// ゲーム起動時に生成
14      /// </summary>
15      [RuntimeInitializeOnLoadMethod]
16      private static void Init()
17      {
18          SaveDataManager pref = Resources.Load<SaveDataManager>("Singletons/SaveDataManager");
19          if (pref != null) Instantiate(pref);
20      }
```

- 前回の SceneLoadManager などと**同じ形**
- **Resources/Singletons** の SaveDataManager.prefab を読む
- 起動時に1つ生成 → Awake で **DontDestroyOnLoad**

POINT　**Persistent はすべて「プレハブ＋Init」で統一**する。どこにあるか迷わず、あとから Inspector で設定する項目が増えても困らない。

STEP 3　準備

SaveDataManager プレハブを作る

- ① 空の GameObject「**SaveDataManager**」を作り、SaveDataManager.cs を付ける
- ② **Assets/Resources/Singletons** フォルダへドラッグして**プレハブ化**
- ③ シーンに残った SaveDataManager は**削除**する
- ④ 再生して、Hierarchy の **DontDestroyOnLoad** に SaveDataManager がいることを確認

注意　プレハブ名と **"Singletons/SaveDataManager"** の文字列は**完全に一致**させる。シーンに残すと重複として消される。

STEP 3　ルール

呼んでいい向き・いけない向き

StageTimer
Scene（表示する側）

→ OK

SaveDataManager
Persistent（データを持つ側）

StageTimer の Text
シーンと一緒に消える

← NG

SaveDataManager
シーン内の物を持たない

POINT　**Scene → Persistent** は呼んでよい（相手はずっといる）。**Persistent → Scene** の参照は持たない（相手は消える）。

StageTimer.cs（完成版）

C# Assets/Scripts/StageTimer.cs

```
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;

// ステージのクリアタイムを計る（シーンごとにリセット）
public class StageTimer : SceneSingleton<StageTimer>
{
    [SerializeField] private TextMeshProUGUI timeText;
    [SerializeField] private TextMeshProUGUI bestText;  // 追加

    private float elapsedTime;
    private bool isRunning = true;
```

StageTimer.cs（完成版）

C# Assets/Scripts/StageTimer.cs

```
14      // 追加：ステージ開始時にベストタイムを表示
15      private void Start()
16      {
17          string sceneName = SceneManager.GetActiveScene().name;
18          if (SaveDataManager.Instance.HasBestTime(sceneName))
19          {
20              float best = SaveDataManager.Instance.GetBestTime(sceneName);
21              bestText.text = "BEST " + best.ToString("F2");
22          }
23          else
24          {
25              bestText.text = "BEST --.--";
26          }
27      }
28
29      private void Update()
30      {
31          if (!isRunning) return;
32          elapsedTime += Time.deltaTime;
33          timeText.text = "TIME " + elapsedTime.ToString("F2");
34      }
```

StageTimer.cs（完成版）

C# Assets/Scripts/StageTimer.cs

```
36      public void StopTimer()
37      {
38          isRunning = false;
39      }
40
41      // 追加：クリアしたときに呼ぶ
42      public void RecordClear()
43      {
44          StopTimer();
45          string sceneName = SceneManager.GetActiveScene().name;
46          if (SaveDataManager.Instance.TryUpdateBestTime(sceneName, elapsedTime))
47          {
48              bestText.text = "NEW RECORD!";
49          }
50      }
51  }
```

STEP 3　解説

Start でベストタイムを表示する

C# Assets/Scripts/StageTimer.cs

```
14      // 追加：ステージ開始時にベストタイムを表示
15      private void Start()
16      {
17          string sceneName = SceneManager.GetActiveScene().name;
18          if (SaveDataManager.Instance.HasBestTime(sceneName))
19          {
20              float best = SaveDataManager.Instance.GetBestTime(sceneName);
21              bestText.text = "BEST " + best.ToString("F2");
22          }
23          else
24          {
25              bestText.text = "BEST --.--";
26          }
27      }
```

ポイント

- Scene → Persistent の向きで**読みに行く**
- Awake ではなく **Start**
- 記録がなければ「--.--」

STEP 3　解説

RecordClear — 止めて、記録を渡す

C# Assets/Scripts/StageTimer.cs

```
41      // 追加：クリアしたときに呼ぶ
42      public void RecordClear()
43      {
44          StopTimer();
45          string sceneName = SceneManager.GetActiveScene().name;
46          if (SaveDataManager.Instance.TryUpdateBestTime(sceneName, elapsedTime))
47          {
48              bestText.text = "NEW RECORD!";
49          }
50      }
```

- まず **StopTimer()**
- **TryUpdateBestTime** が true なら「NEW RECORD!」
- 記録するのは**クリアしたときだけ**

STEP 3　書き換え

Player.GameClear() を RecordClear に変える

C# Assets/Scripts/Player.cs

```
171     public void GameClear()
172     {
173         if (isGameEnd) return;
174         isGameEnd = true;
175         anim.SetFloat("Speed", 0f);
176         InGameUi.Instance.GameClear();
177         StageTimer.Instance.RecordClear();  // 追加
178     }
```

ポイント

- STEP 2 の **StopTimer()** を **RecordClear()** に
- GameOver の方は StopTimer() のまま

STEP 3　動かしてみよう

ここまでで確かめること

1. 初めてクリア → **NEW RECORD!**
2. 同じステージをもう一度 → 最初に **BEST** が表示されている
3. 遅くクリア → 記録は**変わらない**／速くクリア → **NEW RECORD!**
4. Hierarchy：**DontDestroyOnLoad** の中に SaveDataManager がいる

CHALLENGE

PauseMenu

4

Esc でポーズ。自分で判断して作ってみる

チャレンジ

Esc キーでポーズする PauseMenu を作る

- Esc で**止める／再開**（Time.timeScale を 0 ／ 1）
- 止めている間は「PAUSE」の**パネルを表示**（Canvas に Panel を追加）
- **先に答える**：Scene と Persistent のどちら？　理由は 2つの質問で

注意　ヒント：ポーズ中に Enter でリトライやタイトルに戻ったら、次のシーンは**どうなる**？

PauseMenu.cs（解答例）

C# Assets/Scripts/PauseMenu.cs

```
using UnityEngine;

// Esc キーでポーズする（シーンに置く）
public class PauseMenu : SceneSingleton<PauseMenu>
{
    [SerializeField] private GameObject pausePanel;

    public bool IsPaused { get; private set; }

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            SetPause(!IsPaused);
        }
    }
```

全文　2 / 2

PauseMenu.cs（解答例）

C# Assets/Scripts/PauseMenu.cs

```
18      public void SetPause(bool pause)
19      {
20          IsPaused = pause;
21          pausePanel.SetActive(pause);
22          Time.timeScale = pause ? 0f : 1f;
23      }
24
25      protected override void OnDestroy()
26      {
27          base.OnDestroy();
28          Time.timeScale = 1f;  // シーンを出るときは必ず元に戻す
29      }
30  }
```

チャレンジ　解説

シーンに置いた物でも、後始末は必要

C# Assets/Scripts/PauseMenu.cs

```
25      protected override void OnDestroy()
26      {
27          base.OnDestroy();
28          Time.timeScale = 1f;  // シーンを出るときは必ず元に戻す
29      }
```

ポイント

- **Time.timeScale** は static ＝ ゲーム全体で1つ
- PauseMenu が消えても**0 のまま残る**
- だから消えるときに**元に戻す**
- **base.OnDestroy()** も忘れずに

もっとやるなら

宿題・発展

① タイトルにベストタイム一覧

Title シーンに Text を置き、**SaveDataManager** から読んで表示する。
どちら向きの呼び出しか説明できるように。

② 本当にセーブする

今はゲームを閉じると消える。
SaveDataManager に **Save / Load** を足し、**PlayerPrefs** に保存する。